

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

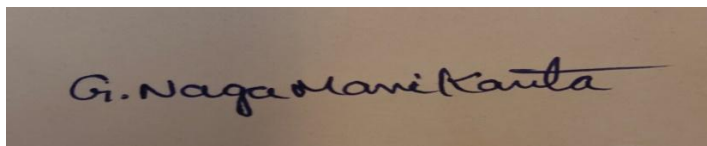
Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I G. Naga Manikant (192425310) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student :G.Naga Manikanta

QUESTIONS:**Total Marks:25****1. Error Analysis in Maximum Likelihood Estimation (MLE)**

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- a) Identify the conceptual error in the likelihood formulation.
- b) Explain how this mistake affects parameter estimation.
- c) Suggest the correct mathematical formulation and reasoning.
- d) How would using log-likelihood fix computational issues?

ANSWER:**a) Identify the conceptual error in the likelihood formulation.**

The student incorrectly writes the **likelihood as the sum of probabilities** instead of the **product of probabilities**.

For independent observations, the likelihood should be:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The student's incorrect formulation is:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

This is conceptually wrong because the joint probability of independent observations is obtained by **multiplying** their probabilities, not adding them.

b) Explain how this mistake affects parameter estimation.

Using the sum of probabilities does not represent the true likelihood of the observed data. As a result:

- The optimization objective becomes incorrect.
- The estimated parameters do not maximize the true probability of the dataset.
- The classifier learns inaccurate parameters.
- Even with a large amount of data, the model produces poor predictions because it is optimizing the wrong function.

c) Suggest the correct mathematical formulation and reasoning.

The correct likelihood function is

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

where:

- $P(x_i | \theta)$ is the probability of observing sample x_i .
- The product is used because all training samples are assumed to be **independent**.

The Maximum Likelihood Estimation (MLE) chooses the parameter θ that maximizes this likelihood.

d) How would using log-likelihood fix computational issues?

Instead of maximizing the product of probabilities, we maximize the **log-likelihood**:

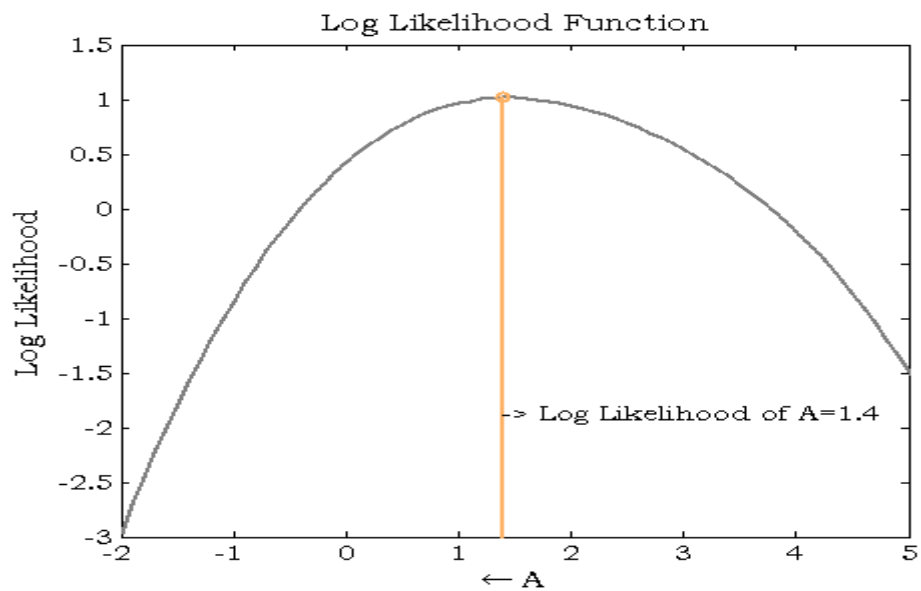
$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Using log-likelihood provides several advantages:

- Converts the product into a sum, making calculations easier.
 - Prevents numerical underflow caused by multiplying many small probabilities.
 - Simplifies differentiation and optimization during parameter estimation.
 - Produces the same optimal parameter values because the logarithm is a monotonic function.
-

Conclusion

The student's mistake is using the **sum of probabilities instead of their product** to define the likelihood function. This leads to incorrect parameter estimation and poor classifier performance. The correct likelihood is the **product of probabilities**, and in practice, the **log-likelihood** is maximized because it is mathematically simpler, numerically stable, and computationally efficient.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- List possible implementation errors in the learning algorithm.

- b) Analyze how incorrect learning rate or weight updates affect convergence.
- c) Identify dataset-related issues that may cause this problem.
- d) Propose modifications to ensure convergence.

ANSWER:

a) List possible implementation errors in the learning algorithm.

The Perceptron may fail to converge due to the following implementation errors:

- Incorrect weight update formula.
 - Bias term is not included or updated.
 - Wrong prediction function or activation function.
 - Learning rate is set incorrectly.
 - Errors in data preprocessing or feature scaling.
 - Updating weights even for correctly classified samples.
-

b) Analyze how incorrect learning rate or weight updates affect convergence.

- **Very high learning rate:** The weights change too much in each iteration, causing the model to overshoot the correct solution and oscillate.
- **Very low learning rate:** The weights change very slowly, resulting in slow convergence.
- **Incorrect weight update rule:** If the update formula is implemented incorrectly, the weights move in the wrong direction, preventing convergence.

The correct Perceptron update rule is:

$$w_{\text{new}} = w_{\text{old}} + \eta(y - \hat{y})x$$

where:

- η = learning rate
 - y = actual class
 - $\hat{y}$ = predicted class
 - x = input feature vector
-

c) Identify dataset-related issues that may cause this problem.

Possible dataset issues include:

- Data is **not actually linearly separable**.
- Incorrect or noisy class labels.
- Presence of outliers.
- Features are not properly normalized or scaled.

- Duplicate or inconsistent training samples.
-

d) Propose modifications to ensure convergence.

To ensure convergence:

- Use the correct Perceptron weight update rule.
 - Choose a suitable learning rate (e.g., 0.01 or 0.1).
 - Include and update the bias term.
 - Normalize or standardize the input features.
 - Remove noisy or incorrectly labeled samples.
 - Verify that the dataset is linearly separable before training.
-

Conclusion

A Perceptron should converge on a **linearly separable dataset**. Failure to converge usually occurs because of implementation mistakes, incorrect learning rate, improper weight updates, or dataset problems such as noise or non-linearity. Using the correct update rule, appropriate learning rate, bias term, and clean, normalized data helps ensure successful convergence.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- a) Identify possible mathematical or coding errors in gradient computation.
- b) Explain the role of activation functions in gradient flow.
- c) Analyze how vanishing or exploding gradients affect training.
- d) Suggest techniques to fix the issue (e.g., normalization, initialization).

ANSWER:

A Multilayer Perceptron (MLP) trained using **Backpropagation** should show decreasing loss during training. If the loss keeps increasing, it usually indicates errors in gradient computation or the training process.

a) Identify possible mathematical or coding errors in gradient computation.

Possible errors include:

- Incorrect derivative of the activation function.
- Wrong application of the chain rule during backpropagation.
- Incorrect weight update formula (adding gradients instead of subtracting them).
- Learning rate is too high.

- Incorrect initialization of weights or bias values.
 - Bugs in matrix multiplication or indexing.
-

b) Explain the role of activation functions in gradient flow.

Activation functions introduce **non-linearity**, allowing the network to learn complex patterns.

- During backpropagation, gradients pass through the activation function.
- If the activation function has very small derivatives (e.g., sigmoid for large inputs), gradients become very small.
- Activation functions like **ReLU** help gradients flow better and speed up learning.

Thus, the choice of activation function directly affects how well gradients propagate through the network.

c) Analyze how vanishing or exploding gradients affect training.

Vanishing Gradients:

- Gradients become extremely small.
- Early layers receive almost no updates.
- Learning becomes very slow or stops completely.

Exploding Gradients:

- Gradients become extremely large.
- Weight updates become unstable.
- Loss increases rapidly, and training may diverge.

Both problems prevent the network from learning effectively.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

To improve training:

- Use **He or Xavier initialization** for weights.
 - Normalize input data using **Batch Normalization**.
 - Replace sigmoid with **ReLU** or similar activation functions.
 - Reduce the learning rate if it is too high.
 - Apply **Gradient Clipping** to prevent exploding gradients.
 - Use adaptive optimizers such as **Adam** or **RMSProp**.
-

Conclusion

Increasing loss during backpropagation is usually caused by incorrect gradient computation, poor activation functions, vanishing/exploding gradients, or improper initialization. Using correct gradient calculations, suitable activation functions, proper weight initialization, normalization, and

adaptive optimization techniques helps stabilize training and ensures that the network learns effectively.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- a) Explain why batch gradient descent is slow in large datasets.
- b) Analyze the cause of fluctuations in SGD.
- c) Compare the error behavior of Mini-Batch Gradient Descent.
- d) Recommend the best approach and justify with error trade-offs.

ANSWER:

a) Explain why Batch Gradient Descent is slow in large datasets.

Batch Gradient Descent computes the gradient using **all training samples** before updating the weights.

- Every iteration processes the entire dataset.
- This requires high computation and memory.
- Training becomes slow, especially for large datasets.

Hence, Batch Gradient Descent converges slowly on large datasets.

b) Analyze the cause of fluctuations in SGD.

Stochastic Gradient Descent (SGD) updates the weights using **one training sample at a time**.

- Each sample produces a different gradient.
 - Weight updates become noisy.
 - The loss fluctuates instead of decreasing smoothly.
 - Although unstable, these fluctuations can help escape local minima.
-

c) Compare the error behavior of Mini-Batch Gradient Descent.

Mini-Batch Gradient Descent updates weights using a **small batch of samples** (e.g., 32 or 64).

- Loss decreases more smoothly than SGD.
- Faster than Batch Gradient Descent.
- Less noisy than SGD.
- Provides stable and efficient convergence.

Thus, it offers a balance between speed and stability.

d) **Recommend the best approach and justify with error trade-offs.**

Mini-Batch Gradient Descent is generally the best choice because it combines the advantages of both methods.

Method	Error Behavior
Batch Gradient Descent	Smooth but slow convergence.
Stochastic Gradient Descent (SGD)	Fast updates but noisy and fluctuating loss.
Mini-Batch Gradient Descent	Faster than Batch GD and more stable than SGD.

Mini-Batch Gradient Descent provides:

- Faster training.
- Stable convergence.
- Better computational efficiency.
- Good generalization performance.

Conclusion

Batch Gradient Descent is slow because it processes the entire dataset before each update. SGD is faster but shows unstable loss due to noisy updates from individual samples. Mini-Batch Gradient Descent balances both approaches by using small batches, resulting in faster training, smoother convergence, and better overall performance. Therefore, **Mini-Batch Gradient Descent is the recommended approach** for most machine learning applications.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

ANSWER:

a) Explain how high dimensionality increases model error.

High-dimensional data contains a large number of features, making the model more complex.

- The model learns unnecessary patterns and noise.
- More training data is required to learn accurately.
- The model overfits the training data.

- As a result, the **test error increases** even though the training error is low.
-

b) Identify signs of overfitting in this scenario.

The following signs indicate overfitting:

- Very high accuracy on the training dataset.
 - Poor accuracy on the test dataset.
 - Large gap between training and testing performance.
 - The model memorizes training data instead of learning general patterns.
-

c) Analyze why distance-based learning fails in high dimensions.

Distance-based algorithms such as **K-Nearest Neighbors (KNN)** rely on measuring distances between data points.

In high dimensions:

- Data points become sparse.
 - Distances between points become very similar.
 - It becomes difficult to identify the true nearest neighbors.
 - As a result, the model makes inaccurate predictions.
-

d) Suggest dimensionality reduction or regularization techniques to improve performance.

To improve performance:

- Use **Principal Component Analysis (PCA)** to reduce the number of features.
 - Apply **Feature Selection** to remove irrelevant features.
 - Use **L1 (Lasso)** or **L2 (Ridge)** regularization to reduce overfitting.
 - Collect more training data if possible.
 - Use cross-validation to select the best model.
-

Conclusion

The Curse of Dimensionality increases model complexity and leads to overfitting, causing poor performance on unseen data. Distance-based algorithms also become less effective because distances lose their meaning in high-dimensional spaces. Applying **dimensionality reduction techniques like PCA** and **regularization methods such as L1/L2** helps reduce overfitting and improves the model's ability to generalize.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand